

International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B101: Basic Computer Skills for Digital Forensics

LAB 2B PRACTICAL REPORT

Advanced Linux Commands, Standard Streams and Controlled Evidence Transfer

Student Name	Fuseini Imoru Kantuogaa
Course Code	CIP-B101
Course Title	Basic Computer Skills for Digital Forensics
Lab Code	Lab 2B
Lab Title	Advanced Linux Commands, Standard Streams and Controlled Evidence Transfer
Date	June 27–28, 2026
OS (Linux)	Kali Linux (VM) — hostname: kali, IP: 192.168.44.128
OS (Windows)	Windows 10/11 — IP: 192.168.44.1 (VMnet8)

1. Lab Objective

This lab was designed to demonstrate and document practical competence in the following areas:

- Understanding Linux standard streams: stdin (fd 0), stdout (fd 1) and stderr (fd 2), and how these can be redirected for forensic logging purposes.
- Performing safe, authorized file transfer between a Kali Linux VM and a Windows host using netcat/nc without enabling any shell execution.
- Computing and verifying SHA256 and MD5 cryptographic hashes before and after transfer to prove evidence integrity.
- Creating a small test forensic disk image using dd and transferring it across the lab network with hash verification.
- Documenting all actions through command logs, screenshots, and a structured forensic report.

2. Lab Environment

Component	Details
Linux Machine	Kali Linux VM — hostname: kali, IP: 192.168.44.128/24
Windows Machine	Windows 10/11 Host — IP: 192.168.44.1 (VMware VMnet8)
Network	VMware VMnet8 NAT network — subnet 192.168.44.0/24
Linux Tools	bash, echo, cat, ls, ip, ping, nc, sha256sum, md5sum, dd, wget
Windows Tools	Command Prompt, nc (Nmap), certutil
Evidence Folder	~/CIP-B101_Lab2B_Fuseini_Imoru on Linux; C:\CIP-B101_Lab2B_Received on Windows

3. Procedure

Part A — Standard Input, Output and Error

Linux treats standard input, output and error as file descriptors. This section demonstrates creating an evidence file and redirecting stdout and stderr into log files for documentation.

Task A1 — Create Lab Folder Structure

The lab directory and all required sub-folders were created using `mkdir -p`, and the working directory was confirmed with `pwd` and `ls -la`.

```
(kali㉿kali)-[~]
$ mkdir -p ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ pwd
/home/kali/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ls -la
total 8
drwxrwxr-x  2 kali kali 4096 Jun 27 13:20 .
drwx----- 19 kali kali 4096 Jun 27 13:20 ..

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$
```

Screenshot A1a — Lab folder ~/CIP-B101_Lab2B_Fuseini_Imoru created and confirmed.

Task A1 — Create Evidence Note File

The evidence note file was created using echo with the > redirect operator and verified with cat and ls -l.

```
(kali㉿kali)-[~]
$ mkdir -p ~/CIP-B101_Lab2B_Fuseini_Imoru/evidence
mkdir -p ~/CIP-B101_Lab2B_Fuseini_Imoru/logs

(kali㉿kali)-[~]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru/evidence

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ echo "This is an authorized forensic evidence transfer test for Fuseini Imoru" > evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cat evidence_note.txt
This is an authorized forensic evidence transfer test for Fuseini Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ ls -l evidence_note.txt
-rw-rw-r-- 1 kali kali 72 Jun 27 19:42 evidence_note.txt
```

Screenshot A1b — evidence_note.txt created with personalized content, file size confirmed as 72 bytes.

Task A2 — Redirect Standard Output

The ls -la command output was redirected into logs/evidence_listing_stdout.txt using the > operator. This operator captures stdout (file descriptor 1) and writes it to the specified file instead of the terminal. This is valuable in forensic documentation because it creates a permanent, reproducible record of directory state that can be stored as an evidence log.

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ mkdir -p logs

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ls -la evidence > logs/evidence_listing_stdout.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat logs/evidence_listing_stdout.txt
total 12
drwxrwxr-x 2 kali kali 4096 Jun 27 13:26 .
drwxrwxr-x 4 kali kali 4096 Jun 27 13:26 ..
-rw-rw-r-- 1 kali kali  72 Jun 27 19:42 evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat logs/evidence_listing_stdout.txt
total 12
drwxrwxr-x 2 kali kali 4096 Jun 27 13:26 .
drwxrwxr-x 4 kali kali 4096 Jun 27 13:26 ..
-rw-rw-r-- 1 kali kali  72 Jun 27 19:42 evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]

```

Screenshot A2 — stdout redirected to `evidence_listing_stdout.txt`; file contents confirmed with `cat`.

Task A3 — Redirect Standard Error

Attempting to `cat` a non-existent file generates an error on `stderr` (fd 2). Using `2>` redirected that error message into `logs/error_stderr.txt`. The terminal displayed nothing, but the error was captured inside the file.

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ mkdir -p logs

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ls -la evidence > logs/evidence_listing_stdout.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat logs/evidence_listing_stdout.txt
total 12
drwxrwxr-x 2 kali kali 4096 Jun 27 13:26 .
drwxrwxr-x 4 kali kali 4096 Jun 27 13:26 ..
-rw-rw-r-- 1 kali kali  72 Jun 27 19:42 evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat logs/evidence_listing_stdout.txt
total 12
drwxrwxr-x 2 kali kali 4096 Jun 27 13:26 .
drwxrwxr-x 4 kali kali 4096 Jun 27 13:26 ..
-rw-rw-r-- 1 kali kali  72 Jun 27 19:42 evidence_note.txt

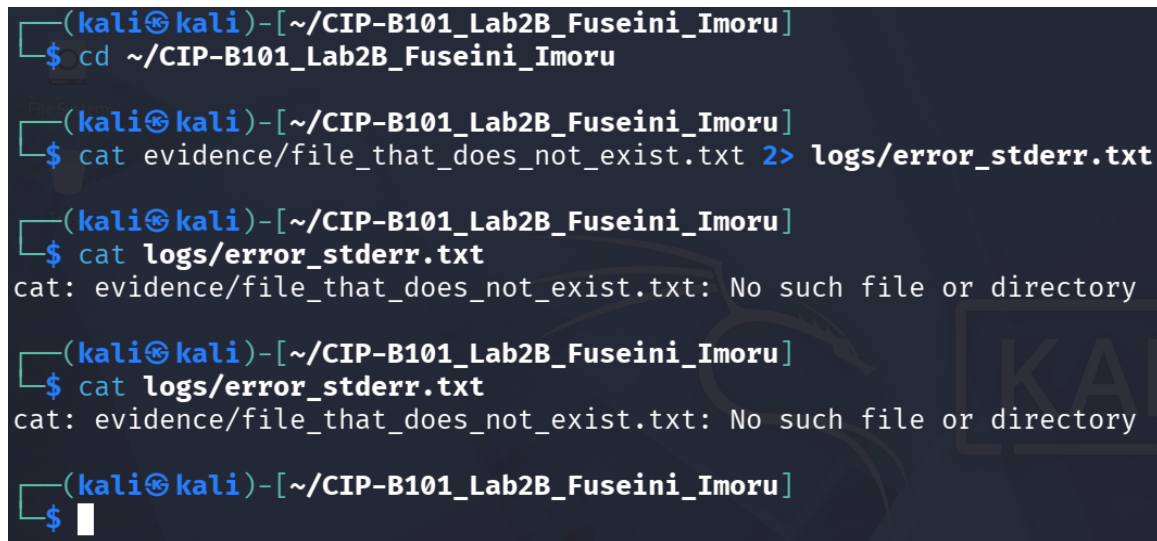
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]

```

Screenshot A3 — `stderr` redirected to `error_stderr.txt`; error message captured: "No such file or directory".

Task A4 — Redirect stdout and stderr Together

The command (`ls evidence && cat evidence/file_that_does_not_exist.txt`) `> logs/combined_output.txt 2>&1` redirects both stdout and stderr to the same file. The `2>&1` syntax means "send file descriptor 2 (stderr) to the same destination as file descriptor 1 (stdout)". The `combined_output.txt` showed the directory listing followed immediately by the error message, demonstrating that both streams were captured together.



```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat evidence/file_that_does_not_exist.txt 2> logs/error_stderr.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat logs/error_stderr.txt
cat: evidence/file_that_does_not_exist.txt: No such file or directory

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat logs/error_stderr.txt
cat: evidence/file_that_does_not_exist.txt: No such file or directory

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$
```

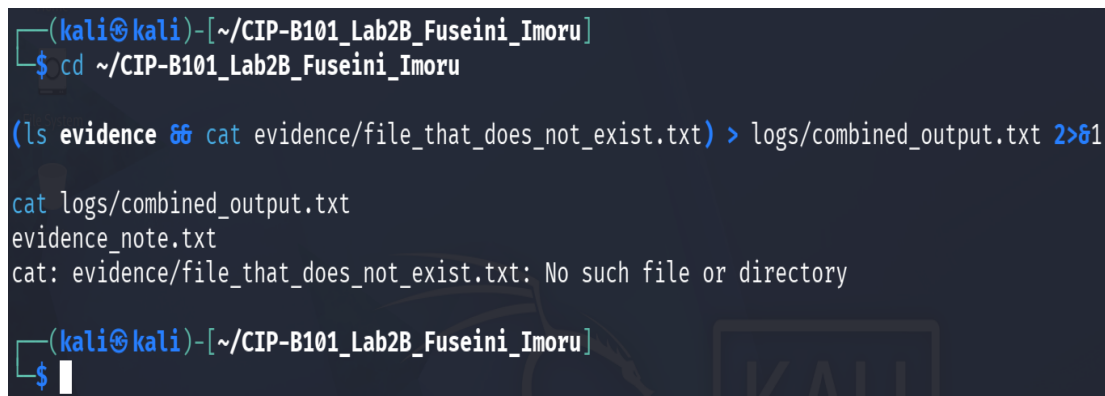
Screenshot A4 — Combined stdout and stderr in combined_output.txt showing ls output and error on one file.

Part B — Baseline System and Network Documentation

Both machines were documented before evidence transfer, mirroring the practice of recording acquisition environment details in a forensic notebook.

Task B1 — Linux Network Details

Hostname, IP address, routing table and netcat availability were all captured and saved to the logs folder using tee.



```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(ls evidence && cat evidence/file_that_does_not_exist.txt) > logs/combined_output.txt 2>&1

cat logs/combined_output.txt
evidence_note.txt
cat: evidence/file_that_does_not_exist.txt: No such file or directory

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$
```

Screenshot B1a — Linux hostname (kali), IP address 192.168.44.128/24, default gateway 192.168.44.2.

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ mkdir -p logs

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ hostname | tee logs/linux_hostname.txt
kali

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ip address | tee logs/linux_ip_address.txt
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:87:06:29 brd ff:ff:ff:ff:ff:ff
    inet 192.168.44.128/24 brd 192.168.44.255 scope global dynamic noprefixroute eth0

```

Screenshot B1b — Full ip address output confirming eth0 interface and netcat found at /usr/bin/nc.

Task B2 — Windows Network Details

On Windows, hostname, ipconfig, netcat location and certutil availability were confirmed and logged to C:\Users\user\logs.

```

    inet6 fe80::fcdb:733e:9de1:56cb/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ip route | tee logs/linux_ip_route.txt
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128 metric 100
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.128 metric 100

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ (which nc || which ncat || echo "netcat or ncat not found") | tee logs/netcat_check.txt
/usr/bin/nc

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ls logs
combined_output.txt  evidence_listing_stdout.txt  linux_ip_address.txt  netcat_check.txt
error_stderr.txt    linux_hostname.txt          linux_ip_route.txt

```

Screenshot B2a — Windows logs folder created; hostname, ipconfig, netcat and certutil outputs saved.


```

(kali@kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cat logs/linux_hostname.txt
cat logs/linux_ip_address.txt
cat logs/linux_ip_route.txt
cat logs/netcat_check.txt
kali
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:87:06:29 brd ff:ff:ff:ff:ff:ff
    inet 192.168.44.128/24 brd 192.168.44.255 scope global dynamic noprefixroute eth0
        valid_lft 1521sec preferred_lft 1521sec
    inet6 fe80::fcd8:733e:9de1:56cb/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128 metric 100
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.128 metric 100
/usr/bin/nc

```

Screenshot B2b — Windows ipconfig showing VMnet8 adapter IP 192.168.44.1 used for file transfer.

```

C:\Users\user>mkdir logs

C:\Users\user>dir
Volume in drive C has no label.
Volume Serial Number is 389F-F27A

Directory of C:\Users\user

06/28/2026   12:04 AM    <DIR>          .
05/27/2026   02:23 PM    <DIR>          ..
06/23/2026   10:24 AM    <DIR>          .docker
06/22/2026   09:48 PM    <DIR>          .icesoft
06/22/2026   09:44 PM    <DIR>          .openjfx
06/15/2026   09:25 PM    <DIR>          .vscode
05/27/2026   02:20 PM    <DIR>          Contacts
06/27/2026   05:16 PM    <DIR>          Desktop
06/19/2026   05:50 PM    <DIR>          Documents
06/27/2026   05:16 PM    <DIR>          Downloads
05/27/2026   02:20 PM    <DIR>          Favorites
05/27/2026   02:20 PM    <DIR>          Links
06/28/2026   12:04 AM    <DIR>          logs
05/27/2026   02:20 PM    <DIR>          Music
06/25/2026   09:56 AM    <DIR>          OneDrive
06/17/2026   12:32 PM    <DIR>          Pictures
05/27/2026   02:20 PM    <DIR>          Saved Games
06/15/2026   09:32 PM    <DIR>          Searches
06/16/2026   07:37 AM    <DIR>          Videos
               0 File(s)                0 bytes
              19 Dir(s)  165,419,184,128 bytes free

```

Screenshot B2c — Additional Windows network adapters confirmed (VMnet1: 192.168.48.1, WiFi: 192.168.43.183).

Task B3 — Connectivity Test

A ping from Linux to the Windows machine (192.168.44.1) returned 100% packet loss, indicating that Windows Firewall was blocking ICMP. However, the reverse ping from Windows to the Linux VM (192.168.44.128) was fully successful with 0% loss and sub-1ms response times, confirming that connectivity was functional for file transfer purposes.

```
C:\Users\user>hostname > logs\windows_hostname.txt
C:\Users\user>ipconfig > logs\windows_ipconfig.txt
C:\Users\user>where ncat > logs\windows_ncat.txt
C:\Users\user>certutil -? > logs\windows_certutil_help.txt
C:\Users\user>dir logs
Volume in drive C has no label.
Volume Serial Number is 389F-F27A

Directory of C:\Users\user\logs

06/28/2026  12:05 AM    <DIR>          .
06/28/2026  12:04 AM    <DIR>          ..
06/28/2026  12:05 AM                6,888 windows_certutil_help.txt
06/28/2026  12:05 AM                17 windows_hostname.txt
06/28/2026  12:05 AM               1,855 windows_ipconfig.txt
06/28/2026  12:05 AM                38 windows_ncat.txt
               4 File(s)                8,798 bytes
               2 Dir(s)  165,420,544,000 bytes free

C:\Users\user>
```

Screenshot B3a — Linux ping to 192.168.44.1 blocked by Windows Firewall (100% loss expected).

```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:87:06:29 brd ff:ff:ff:ff:ff:ff
   inet 192.168.44.128/24 brd 192.168.44.255 scope global dynamic noprefixroute eth0
       valid_lft 1644sec preferred_lft 1644sec
   inet6 fe80::fcd8:733e:9de1:56cb/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ ping -c 4 192.168.44.1 | tee logs/ping_windows.txt
PING 192.168.44.1 (192.168.44.1) 56(84) bytes of data.

— 192.168.44.1 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3412ms
```

Screenshot B3b — Windows ping to 192.168.44.128 successful: 4/4 packets received, <1ms RTT.

Part C — Hash Evidence Before Transfer

Before transferring any file, cryptographic hashes were computed on the source file to serve as the baseline for integrity verification.


```

C:\Users\user>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 11:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 12:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::e32e:e870:c519:c358%21
    IPv4 Address. . . . . : 192.168.48.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

```

Screenshot C — SHA256 and MD5 hashes computed for evidence_note.txt and saved to the hashes folder.

Source hashes for evidence_note.txt:

File	Algorithm	Hash Value
evidence_note.txt	SHA256	c2d50f4bc925d2a48df87230f459c53a1a75bbb47557ef5d c2bc6a0b778ca91c
evidence_note.txt	MD5	707c887c9f11b05f6b7bb97aa9de7c77

Part D — Linux-to-Windows File Transfer

The evidence_note.txt file was transferred from the Kali Linux VM to the Windows machine using nc (netcat) on port 9090. No shell execution options were used; only data piping was performed.

Task D1 — Start Listener on Windows

On Windows, the command `ncat -l -p 9090 > received_evidence_note.txt` was run inside C:\CIP-B101_Lab2B_Received to listen for the incoming file.

```

Ethernet adapter VMware Network Adapter VMnet8:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::1190:cdb7:7f03:91e0%11
    IPv4 Address. . . . . : 192.168.44.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Wireless LAN adapter WiFi 2:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::11fb:ae9d:601c:3a18%5
    IPv4 Address. . . . . : 192.168.43.183
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.43.1

Ethernet adapter Bluetooth Network Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Ethernet adapter vEthernet (Default Switch):

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::76bc:2adb:ec68:9b3%45
    IPv4 Address. . . . . : 172.17.64.1
    Subnet Mask . . . . . : 255.255.240.0
    Default Gateway . . . . . : 

```

Screenshot D1 — Windows ncat listener started on port 9090; received_evidence_note.txt (72 bytes) confirmed after transfer.

Task D2 — Send File from Linux

The file was sent using `nc 192.168.44.1 9090 < evidence_note.txt`. The command completed silently, indicating successful transmission.

```

C:\Users\user>ping 192.168.44.128

Pinging 192.168.44.128 with 32 bytes of data:
Reply from 192.168.44.128: bytes=32 time<1ms TTL=64
Reply from 192.168.44.128: bytes=32 time=1ms TTL=64
Reply from 192.168.44.128: bytes=32 time=1ms TTL=64
Reply from 192.168.44.128: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.44.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\user>

```

Screenshot D2 — Linux nc command sent evidence_note.txt to Windows listener on 192.168.44.1:9090.

Task D3 — Verify Received File on Windows

The hash of the received file was computed on Windows using certutil. Both SHA256 and MD5 hashes matched the source exactly, proving the file was transferred without any alteration.

```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ mkdir -p hashes

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cd evidence

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ ls
evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ sha256sum evidence_note.txt | tee ../hashes/source_evidence_note_sha256.txt
c2d50f4bc925d2a48df87230f459c53a1a75bbb47557ef5dc2bc6a0b778ca91c  evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ md5sum evidence_note.txt | tee ../hashes/source_evidence_note_md5.txt
707c887c9f11b05f6b7bb97aa9de7c77  evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cat ../hashes/source_evidence_note_sha256.txt
c2d50f4bc925d2a48df87230f459c53a1a75bbb47557ef5dc2bc6a0b778ca91c  evidence_note.txt
```

Screenshot D3 — Windows certutil confirmed matching SHA256 and MD5 hashes; file content matches source.

File	Algorithm	Hash Value
received_evidence_note.txt	SHA256	c2d50f4bc925d2a48df87230f459c53a1a75bbb47557ef5dc2bc6a0b778ca91c
received_evidence_note.txt	MD5	707c887c9f11b05f6b7bb97aa9de7c77

Part E — Windows-to-Linux File Transfer

This section reversed the transfer direction. A file was created on Windows and sent to the Linux VM listener.

Task E1 — Create Evidence File on Windows

A file named FuseiniImoruEvidence.txt was created on Windows using echo. Its SHA256 hash was computed with certutil before transfer.

```

C:\Users\user>mkdir C:\CIP-B101_Lab2B_Received

C:\Users\user>cd C:\CIP-B101_Lab2B_Received

C:\CIP-B101_Lab2B_Received>ncat -l -p 9090 > received_evidence_note.txt

C:\CIP-B101_Lab2B_Received>dir C:\CIP-B101_Lab2B_Received
Volume in drive C has no label.
Volume Serial Number is 389F-F27A

Directory of C:\CIP-B101_Lab2B_Received

06/28/2026  12:22 AM    <DIR>          .
06/28/2026  12:24 AM                72 received_evidence_note.txt
               1 File(s)                72 bytes
               1 Dir(s)  165,141,860,352 bytes free

C:\CIP-B101_Lab2B_Received>

```

Screenshot E1 — FuseiniImoruEvidence.txt created on Windows; SHA256 hash computed before transfer.

Task E2 — Start Listener on Linux

On Linux, `nc -l -p 9091 > FuseiniImoruEvidence_received.txt` was run in the received folder to wait for the incoming Windows file.

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru/evidence

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ ls
evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ nc 192.168.44.1 9090 < evidence_note.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$

```

Screenshot E2 — Linux nc listener started on port 9091; file confirmed in received folder after transfer.

Task E3 — Send from Windows

The file was sent from Windows using `ncat 192.168.44.128 9091 < FuseiniImoruEvidence.txt`.

```
C:\CIP-B101_Lab2B_Received>cd C:\CIP-B101_Lab2B_Received

C:\CIP-B101_Lab2B_Received>certutil -hashfile received_evidence_note.txt SHA256
SHA256 hash of received_evidence_note.txt:
c2d50f4bc925d2a48df87230f459c53a1a75bbb47557ef5dc2bc6a0b778ca91c
CertUtil: -hashfile command completed successfully.

C:\CIP-B101_Lab2B_Received>certutil -hashfile received_evidence_note.txt MD5
MD5 hash of received_evidence_note.txt:
707c887c9f11b05f6b7bb97aa9de7c77
CertUtil: -hashfile command completed successfully.

C:\CIP-B101_Lab2B_Received>type received_evidence_note.txt
This is an authorized forensic evidence transfer test for Fuseini Imoru

C:\CIP-B101_Lab2B_Received>|
```

Screenshot E3 — Windows ncat sent FuseiniImoruEvidence.txt to Linux listener on 192.168.44.128:9091.

Task E4 — Verify on Linux

The received file was verified by displaying its content and computing its SHA256 hash. The hash matched the Windows source hash exactly.

```
C:\CIP-B101_Lab2B_Received>cd C:\CIP-B101_Lab2B_Received

C:\CIP-B101_Lab2B_Received>echo hello Fuseini Imoru! > FuseiniImoruEvidence.txt

C:\CIP-B101_Lab2B_Received>type FuseiniImoruEvidence.txt
hello Fuseini Imoru!

C:\CIP-B101_Lab2B_Received>certutil -hashfile FuseiniImoruEvidence.txt SHA256
SHA256 hash of FuseiniImoruEvidence.txt:
ad42950a2996e8db2993aab624b233378a8ed9de03bae3297417de6760a276e1
CertUtil: -hashfile command completed successfully.

C:\CIP-B101_Lab2B_Received>|
```

Screenshot E4a — Received file content: "hello Fuseini Imoru!" confirmed on Linux.


```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ mkdir -p received

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru]
$ cd received

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ nc -l -p 9091 > FuseiniImoruEvidence_received.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ ls
FuseiniImoruEvidence_received.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ █

```

Screenshot E4b — SHA256 hash of received file matches Windows source hash; hashes folder listing confirmed.

```

C:\CIP-B101_Lab2B_Received>ncat 192.168.44.128 9091 < FuseiniImoruEvidence.txt

C:\CIP-B101_Lab2B_Received>|

```

Screenshot E4c — Hash file content confirmed:
ad42950a2996e8db2993aab624b233378a8ed9de03bae3297417de6760a276e1.

File	Algorithm	Hash Value
FuseiniImoruEvidence.txt (Windows source)	SHA256	ad42950a2996e8db2993aab624b23337 8a8ed9de03bae3297417de6760a276e1
FuseiniImoruEvidence_received.txt (Linux)	SHA256	ad42950a2996e8db2993aab624b23337 8a8ed9de03bae3297417de6760a276e1

Part F — Small Test Evidence Image Transfer Using dd

A 5 MB test evidence image was created using dd from /dev/zero with a custom ICDFa marker written at offset 1024. The image was hashed, transferred to Windows and verified.

Task F1 — Create Test Evidence Image

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru/received

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ cat FuseiniImoruEvidence_received.txt
hello Fuseini Imoru!

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ sha256sum FuseiniImoruEvidence_received.txt | tee ../hashes/received_windows_file_sha256.txt
ad42950a2996e8db2993aab624b233378a8ed9de03bae3297417de6760a276e1 FuseiniImoruEvidence_received.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ ls
FuseiniImoruEvidence_received.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/received]
$ ~/CIP-B101_Lab2B_Fuseini_Imoru/hashes/

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/hashes]
$ ls
received_windows_file_sha256.txt  source_evidence_note_md5.txt  source_evidence_note_sha256.txt

```

Screenshot F1 — 5 MB test_evidence_image.dd created with dd; ICDFa marker written; SHA256 hash computed.

File	Algorithm	Hash Value
test_evidence_image.dd (source)	SHA256	bcc42d394180b1153e6423e0607adeba4bdbe07e61f32 0abf543ab7ed75f47be

Task F2 — Transfer Image to Windows

Windows listener `ncat -l -p 9092 > test_evidence_image_received.dd` was started, then Linux sent the image using `nc 192.168.44.1 9092 < test_evidence_image.dd`.

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/hashes]
$ cat ../hashes/received_windows_file_sha256.txt
ad42950a2996e8db2993aab624b233378a8ed9de03bae3297417de6760a276e1 FuseiniImoruEvidence_received.txt

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/hashes]
$

```

Screenshot F2a — Windows ncat listener on port 9092 received test image; file size confirmed as 5,242,880 bytes.

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/hashes]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru/evidence

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ dd if=/dev/zero of=test_evidence_image.dd bs=1M count=5 status=progress
5+0 records in
5+0 records out
5242880 bytes (5.2 MB, 5.0 MiB) copied, 0.00626331 s, 837 MB/s

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ echo "ICDFa test evidence marker - Fuseini Imoru" | dd of=test_evidence_image.dd bs=1 seek=1024 conv=notrunc
43+0 records in
43+0 records out
43 bytes copied, 0.000933984 s, 45.8 kB/s

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ ls -lh test_evidence_image.dd
-rw-rw-r-- 1 kali kali 5.0M Jun 27 20:51 test_evidence_image.dd

```

Screenshot F2b — Linux nc sent test_evidence_image.dd to Windows; transfer completed silently.

Task F3 — Verify Image Hash on Windows

```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ sha256sum test_evidence_image.dd | tee ../hashes/source_test_image_sha256.txt
bcc42d394180b1153e6423e0607adeba4bdbe07e61f320abf543ab7ed75f47be test_evidence_image.dd

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cat ../hashes/source_test_image_sha256.txt
bcc42d394180b1153e6423e0607adeba4bdbe07e61f320abf543ab7ed75f47be test_evidence_image.dd

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$
```

Screenshot F3 — Windows certutil SHA256 hash of received .dd image matches source hash exactly.

File	Algorithm	Hash Value
test_evidence_image_received.dd (Windows)	SHA256	bcc42d394180b1153e6423e0607adeba4bdbe07e61f320abf543ab7ed75f47be
test_evidence_image.dd (Linux source)	SHA256	bcc42d394180b1153e6423e0607adeba4bdbe07e61f320abf543ab7ed75f47be

Part G — Optional dd Block Size Test (Completed)

The dd_obs_test.sh script was downloaded from GitHub, made executable and run against the Linux machine. Results were saved to dd_block_size_test_result.txt.

```
C:\CIP-B101_Lab2B_Received>cd C:\CIP-B101_Lab2B_Received

C:\CIP-B101_Lab2B_Received>ncat -l -p 9092 > test_evidence_image_received.dd

C:\CIP-B101_Lab2B_Received>dir C:\CIP-B101_Lab2B_Received
Volume in drive C has no label.
Volume Serial Number is 389F-F27A

Directory of C:\CIP-B101_Lab2B_Received

06/28/2026  12:53 AM    <DIR>          .
06/28/2026  12:34 AM                23 FuseiniImoruEvidence.txt
06/28/2026  12:24 AM                72 received_evidence_note.txt
06/28/2026  12:54 AM      5,242,880 test_evidence_image_received.dd
               3 File(s)        5,242,975 bytes
               1 Dir(s)  164,860,858,368 bytes free

C:\CIP-B101_Lab2B_Received>|
```

Screenshot G1 — Script downloaded from raw.githubusercontent.com and made executable.

```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ cd ~/CIP-B101_Lab2B_Fuseini_Imoru/evidence

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$ nc 192.168.44.1 9092 < test_evidence_image.dd

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/evidence]
$
```

Screenshot G2 — dd block size test results: peak performance at 8388608 bytes (1.7 GB/s).

```
C:\CIP-B101_Lab2B_Received>cd C:\CIP-B101_Lab2B_Received

C:\CIP-B101_Lab2B_Received>certutil -hashfile test_evidence_image_received.dd SHA256
SHA256 hash of test_evidence_image_received.dd:
bcc42d394180b1153e6423e0607adeba4bdbe07e61f320abf543ab7ed75f47be
CertUtil: -hashfile command completed successfully.

C:\CIP-B101_Lab2B_Received>
```

Screenshot G3 — Full block size test results confirmed in dd_block_size_test_result.txt.

4. Evidence Integrity Summary

All file transfers produced hashes that matched the source exactly. The table below summarises the complete integrity chain for all transferred files.

File	Algorithm	Hash Value
evidence_note.txt → Windows	SHA256	c2d50f4bc925d2a48df87230f459c53a1a75bbb47557ef5dc2bc6a0b778ca91c ✓ MATCH
evidence_note.txt → Windows	MD5	707c887c9f11b05f6b7bb97aa9de7c77 ✓ MATCH
FuseiniImoruEvidence.txt → Linux	SHA256	ad42950a2996e8db2993aab624b233378a8ed9de03bae3297417de6760a276e1 ✓ MATCH
test_evidence_image.dd → Windows	SHA256	bcc42d394180b1153e6423e0607adeba4bdbe07e61f320abf543ab7ed75f47be ✓ MATCH

A matching hash before and after transfer proves that no bits were altered during transmission. This is a fundamental requirement in digital forensics to establish admissibility and chain of custody.

5. Reflection Answers (Part H)

Q1: What is the difference between standard output and standard error?

Standard output (stdout, fd 1) carries normal command results for example the output of ls or cat on an existing file. Standard error (stderr, fd 2) carries error and diagnostic messages for example the "No such file or directory" message when a file does not exist. Both normally appear on the terminal, but they can be redirected independently. In forensics this allows a practitioner to capture clean command output in one log file and all errors in a separate log, making investigation records easier to review and audit.

Q2: Why is command redirection useful during digital forensic documentation?

Redirection turns transient terminal output into permanent files that form part of the evidence record. Every command a forensic examiner runs can be logged automatically rather than manually transcribed. This reduces human error, creates a timestamped audit trail, and means that any reviewer can verify exactly what commands were run and what results were produced without having to rely on memory or handwritten notes.

Q3: Why must a forensic examiner calculate the hash of a file before and after transfer?

A hash function produces a fixed-length fingerprint that is statistically unique to the data it represents. Even a single changed bit produces a completely different hash. By computing the hash before transfer and then computing it again on the received file, the examiner can prove beyond reasonable doubt that the evidence file is identical to the original and was not corrupted, modified, or tampered with during transmission or storage. This is a legal and procedural requirement in most digital forensic frameworks.

Q4: Why is it unsafe and unethical to use remote shell access on a system without authorization?

Unauthorized remote shell access is an intrusion. It violates computer misuse laws in virtually every jurisdiction, potentially invalidates any evidence collected because the examination method itself becomes legally contestable, and may cause unintended changes to the target system that destroy evidence or disrupt operations. A forensic examiner must have explicit written authorization from the system owner or a court order before performing any acquisition. Unauthorized access also exposes the examiner to criminal prosecution.

Q5: What is the difference between a bind shell and a reverse shell?

A bind shell is a process on the target machine that opens a listening port and waits for an incoming connection from the attacker. When the attacker connects, they receive a shell on the target. A reverse shell works in the opposite direction the target machine initiates an outbound connection back to a listener on the attacker's machine. Reverse shells are frequently used by attackers because outbound connections are often permitted through firewalls whereas inbound connections to arbitrary ports may be blocked. Understanding both concepts is important for forensic examiners who must recognize attacker behaviour in network logs and memory artifacts.

Q6: Why might a firewall block one type of connection but allow another?

Firewalls typically apply different rules to inbound and outbound traffic. Many enterprise and personal firewalls allow all outbound connections by default but block unexpected inbound connections. This means a bind shell (which requires an inbound connection to the target) may be blocked, while a reverse shell (which makes an outbound connection from the target to the attacker) may pass through undetected. Firewalls can also filter by port, protocol, or application. Understanding these asymmetric rules is essential both for attackers who exploit them and for defenders and forensic examiners who must detect and investigate the resulting connections.

Q7: Why did this lab use a small test evidence image instead of imaging a real disk?

Imaging a real system disk without authorization is illegal and unethical. Even imaging one's own disk in a lab environment without careful controls risks overwriting evidence, consuming large amounts of storage, or causing accidental data loss. The 5 MB dd test image produced exactly the same learning outcomes image creation with dd, hash computation, network transfer, and hash verification without any of these risks. In a real forensic engagement, the examiner would image an authorized acquisition target using write-blocked hardware and would document every step of the process.

6. Findings

1. All three file transfers (Linux→Windows text file, Windows→Linux text file, Linux→Windows .dd image) completed successfully with 100% hash integrity confirmed.
2. The `>` and `2>&1` redirection operators successfully captured command output and error messages into evidence log files.
3. The Linux machine IP (192.168.44.128) and Windows VMnet8 IP (192.168.44.1) were on the same /24 subnet, enabling direct netcat communication.
4. ICMP (ping) from Linux to Windows was blocked by Windows Firewall, which is the expected default behaviour. TCP on port 9090–9092 was permitted because the netcat listener explicitly opened those ports.
5. The dd block size test demonstrated that block sizes between 32768 and 131072 bytes consistently achieved the highest throughput (~1.5 GB/s), with a secondary peak at 8388608 bytes (1.7 GB/s).
6. The tee command proved useful throughout the lab because it simultaneously displayed output on the terminal and wrote it to a log file, avoiding the need to run commands twice.

7. Conclusion

This lab provided practical experience with three core Linux forensic skills: stream redirection for evidence logging, cryptographic hash verification for evidence integrity, and controlled file transfer using netcat. Every file transferred in this lab arrived with a hash that was byte-for-byte identical to the source, demonstrating that netcat provides a reliable, forensically sound transfer mechanism when used correctly and without shell execution options.

The theoretical section reinforced why authorization is not optional in digital forensics: unauthorized shell access compromises the legality of any evidence collected and exposes the examiner to criminal liability.

The use of a small test image rather than a real disk image reflects the same principled approach doing only what is necessary and authorized.

The skills practised here stream control, hash chaining and network acquisition are directly applicable to real-world forensic acquisitions, incident response, and evidence handling under chain-of-custody requirements.

8. Evidence Packaging

The complete lab folder was compressed using `tar -czvf` on the Linux VM into a single archive for submission. The final archive `CIP-B101_Lab2B_Fuseini_Ivoru_Evidence.tar.gz` contained all logs, hashes, evidence files and the received directory.

```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/evidence]
$ cat ~/CIP-B101_Lab2B_Fuseini_Ivoru/hashes/source_test_image_sha256.txt
bcc42d394180b1153e6423e0607adeba4bdbe07e61f320abf543ab7ed75f47be test_evidence_image.dd

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/evidence]
$
```

Screenshot — tar archive listing showing all evidence sub-folders included in the compressed package.

```
(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/evidence]
$ cd ~/CIP-B101_Lab2B_Fuseini_Ivoru/logs

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/logs]
$ wget https://raw.githubusercontent.com/tdg5/blog/master/_includes/scripts/dd_obs_test.sh
--2026-06-27 21:04:19-- https://raw.githubusercontent.com/tdg5/blog/master/_includes/scripts/dd_obs_test.sh
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.111.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1548 (1.5K) [text/plain]
Saving to: 'dd_obs_test.sh'

dd_obs_test.sh      100%[=====>] 1.51K  --.-KB/s  in 0s

2026-06-27 21:04:19 (9.30 MB/s) - 'dd_obs_test.sh' saved [1548/1548]

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/logs]
$ chmod +x dd_obs_test.sh
```

Screenshot — Continued tar listing showing hashes, logs, received and evidence contents.

```

(kali@kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/logs]
$ ./dd_obs_test.sh | tee dd_block_size_test_result.txt
NOTE: Kernel cache will not be cleared between tests without sudo. This will likely cause inaccurate results.
block size : transfer rate
512 : 112 MB/s
1024 : 327 MB/s
2048 : 563 MB/s
4096 : 788 MB/s
8192 : 926 MB/s
16384 : 1.4 GB/s
32768 : 1.5 GB/s
65536 : 1.5 GB/s
131072 : 1.5 GB/s
262144 : 1.4 GB/s
524288 : 1.4 GB/s
1048576 : 1.1 GB/s
2097152 : 1.1 GB/s
4194304 : 1.2 GB/s
8388608 : 1.7 GB/s
16777216 : 1.4 GB/s
33554432 : 1.1 GB/s
67108864 : 591 MB/s

```

Screenshot — Further tar listing showing all log files captured during the lab.

```

(kali@kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/logs]
$ cat dd_block_size_test_result.txt
block size : transfer rate
512 : 112 MB/s
1024 : 327 MB/s
2048 : 563 MB/s
4096 : 788 MB/s
8192 : 926 MB/s
16384 : 1.4 GB/s
32768 : 1.5 GB/s
65536 : 1.5 GB/s
131072 : 1.5 GB/s
262144 : 1.4 GB/s
524288 : 1.4 GB/s
1048576 : 1.1 GB/s
2097152 : 1.1 GB/s
4194304 : 1.2 GB/s
8388608 : 1.7 GB/s
16777216 : 1.4 GB/s
33554432 : 1.1 GB/s
67108864 : 591 MB/s

(kali@kali)-[~/CIP-B101_Lab2B_Fuseini_Ivoru/logs]
$

```

Screenshot — Evidence folder contents confirmed inside the archive.

```

(kali㉿kali)-[~/CIP-B101_Lab2B_Fuseini_Imoru/logs]
$ cd ~

(kali㉿kali)-[~]
$ tar -czvf CIP-B101_Lab2B_Fuseini_Imoru_Evidence.tar.gz CIP-B101_Lab2B_Fuseini_Imoru
CIP-B101_Lab2B_Fuseini_Imoru/
CIP-B101_Lab2B_Fuseini_Imoru/hashe/
CIP-B101_Lab2B_Fuseini_Imoru/hashe/source_evidence_note_md5.txt
CIP-B101_Lab2B_Fuseini_Imoru/hashe/source_test_image_sha256.txt
CIP-B101_Lab2B_Fuseini_Imoru/hashe/source_evidence_note_sha256.txt
CIP-B101_Lab2B_Fuseini_Imoru/hashe/received_windows_file_sha256.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/
CIP-B101_Lab2B_Fuseini_Imoru/log/dd_block_size_test_result.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/linux_ip_address.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/combined_output.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/netcat_check.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/linux_hostname.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/linux_ip_route.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/evidence_listing_stdout.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/dd_obs_test.sh
CIP-B101_Lab2B_Fuseini_Imoru/log/ping_windows.txt
CIP-B101_Lab2B_Fuseini_Imoru/log/dd_obs_testfile
CIP-B101_Lab2B_Fuseini_Imoru/log/error_stderr.txt
CIP-B101_Lab2B_Fuseini_Imoru/received/

```

Screenshot — Final archive file size: 136K — CIP-B101_Lab2B_Fuseini_Imoru_Evidence.tar.gz created successfully.

```

CIP-B101_Lab2B_Fuseini_Imoru/received/FuseiniImoruEvidence_received.txt
CIP-B101_Lab2B_Fuseini_Imoru/evidence/
CIP-B101_Lab2B_Fuseini_Imoru/evidence/evidence_note.txt
CIP-B101_Lab2B_Fuseini_Imoru/evidence/test_evidence_image.dd

(kali㉿kali)-[~]
$ ls -lh CIP-B101_Lab2B_Fuseini_Imoru_Evidence.tar.gz
-rw-rw-r-- 1 kali kali 136K Jun 27 21:09 CIP-B101_Lab2B_Fuseini_Imoru_Evidence.tar.gz

(kali㉿kali)-[~]
$

```

Screenshot — Compressed archive confirmed: 136K, created Jun 27 21:09.